

Algoritmi trattati a lezione

Regole: Si ammette questo documento all'esame; è consentito arricchire questo documento con appunti personali. Si ammette una copia di questo documento per studente; **non** si ammette in nessun caso la condivisione dello stesso tra diversi candidati. Inoltre, non si ammettono fotocopie di questo documento che riportino appunti personali di altre persone, e non si ammette alcun altro documento al di fuori di questo. Non si farà nessuna eccezione.

```

proc InsertionSort (A)
  for (j = 2 to A.length)
    {
      key = A[j]
      i = j - 1
      while ((i > 0) and (A[i] > key))
      {
        A[i + 1] = A[i]
        i = i - 1
      }
      A[i + 1] = key
    }

```

```

proc RecursiveBinarySearch (A, low, high, k)
  if (low > high)
    then return nil
  mid = (high + low) / 2
  if (A[mid] = k)
    then return mid
  if (A[mid] < k)
    then return RecursiveBinarySearch(A, mid + 1, high, k)
  if (A[mid] > k)
    then return RecursiveBinarySearch(A, low, mid - 1, k)

```

```

proc SelectionSort (A)
  for (j = 1 to A.length - 1)
    {
      min = j
      for (i = j + 1 to A.length)
      {
        if (A[i] < A[min])
          then min = i
      }
      SwapValue(A, min, j)
    }

```

```

proc Merge (A, p, q, r)
  n1 = q - p + 1
  n2 = r - q
  let L[1, ..., n1] and R[1, ..., n2] be new array
  for (i = 1 to n1) L[i] = A[p + i - 1]
  for (j = 1 to n2) R[j] = A[q + j]
  i = 1
  j = 1
  for (k = p to r)
    {
      if (i ≤ n1)
      {
        then
        {
          if (j ≤ n2)
          {
            if (L[i] ≤ R[j])
              then CopyFromL(i)
            else CopyFromR(j)
          }
          else CopyFromL(i)
        }
      }
      else CopyFromR(j)
    }

```

```

proc MergeSort (A, p, r)
  if (p < r)
    then
    {
      q = [(p + r) / 2]
      MergeSort(A, p, q)
      MergeSort(A, q + 1, r)
      Merge(A, p, q, r)
    }

```

```

proc NaivePatternMatching (T, P)
  {
    n = T.length
    m = P.length
    for (s = 0 to n - m)
    {
      if (P[1, ..., m] = T[s + 1, s + m])
      then Print(s)
    }
  }

```

```

proc Partition (A, p, r)
  {
    x = A[r]
    i = p - 1
    for (j = p to r - 1)
    {
      if (A[j] ≤ x)
      then
      {
        i = i + 1
        SwapValue(A, i, j)
      }
    }
    SwapValue(A, i + 1, r)
    return i + 1
  }

```

```

proc QuickSort (A, p, r)
  if (p < r)
    then
    {
      q = Partition(A, p, r)
      QuickSort(A, p, q - 1)
      QuickSort(A, q + 1, r)
    }

```

```

proc RandomizedPartition (A, p, r)
  {
    s = p + Random() * (r - p + 1)
    SwapValue(A, s, r)
    x = A[r]
    i = p - 1
    for (j = p to r - 1)
    {
      if (A[j] ≤ x)
      then
      {
        i = i + 1
        SwapValue(A, i, j)
      }
    }
    SwapValue(A, i + 1, r)
    return i + 1
  }

```

```

proc RandomizedQuickSort (A, p, r)
  if (p < r)
    then
    {
      q = RandomizedPartition(A, p, r)
      RandomizedQuickSort(A, p, q - 1)
      RandomizedQuickSort(A, q + 1, r)
    }

```

```

proc RabinKarpPatternMatching (T, P, d, q)
  {
    n = T.length
    m = P.length
    h = dm-1 mod q
    p = 0
    t0 = 0
    for (i = 1 to m)
    {
      p = (d * p + P[i]) mod q
      t0 = (d * t0 + P[i]) mod q
    }
    for (s = 0 to n - m)
    {
      if (p = t_s)
      then
      {
        if (P[1, ..., m] = T[s + 1, s + m])
        then Print(s)
      }
      if (s < n - m)
      then ts+1 = (d * (t_s - T[s + 1] * h) + T[s + m + 1]) mod q
    }
  }

```

```

proc ComputeDelta (P, Σ)
  m = P.length
  for (q = 0 to m)
    for (x ∈ Σ)
      {
        k = min(m + 1, q + 2)
        repeat
          k = k - 1
        until (P_k ⊆ P_q ∘ x) ⇒ P_k suffisso di P_q ∘ a
        δ(q, a) = k
      }
  return δ

```

```

proc DFAPatternMatching (T, δ, m)
  n = T.length
  q = 0
  for i = 1 to n
    {
      q = δ(q, T[i])
      if (q = m)
        then Print(i - m)
    }

```

```

proc Empty (S)
  if (S.top = 0)
    then return true
  return false

```

```

proc Push (S, x)
  if (S.top = S.max)
    then return "overflow"
  S.top = S.top + 1
  S.VAL[S.top] = x

```

```

proc Pop (S)
  if (Empty(S))
    then return "underflow"
  S.top = S.top - 1
  return S.VAL[S.top + 1]

```

```

proc Enqueue (Q, x)
  if (Q.dim = Q.length)
    then return "overflow"
  Q.VAL[Q.tail] = x
  if (Q.tail = Q.length)
    then Q.tail = 1
  else Q.tail = Q.tail + 1
  Q.dim = Q.dim + 1

```

```

proc Dequeue (Q)
  if (Q.dim = 0)
    then return "underflow"
  x = Q.VAL[Q.head]
  if (Q.head = Q.length)
    then Q.head = 1
  else Q.head = Q.head + 1
  Q.dim = Q.dim - 1
  return x

```

```

proc Empty (S)
  if (S.head = nil)
    then return true
  return false

```

```

proc Push (S, x)
  {Insert(S, x)}

```

```

proc Pop (S)
  if (Empty(S))
    then return "underflow"
  x = S.head
  Delete(S, x)
  return x.key

```

```

proc Empty (Q)
  if (Q.head = nil)
    then return true
  return false

```

```

proc Enqueue (Q, x)
  {Insert(Q, x)}

```

```

proc Dequeue (Q)
  if (Empty(Q))
    then return "underflow"
  x = Q.tail
  Q.tail = x.prev
  Delete(Q, x)
  return x.key

```

```

proc Parent (i)
  {return ⌊ i/2 ⌋}

```

```

proc Left (i)
  {return 2 · i}

```

```

proc Right (i)
  {return 2 · i + 1}

```

```

proc MinHeapify (H, i)
  l = Left(i)
  r = Right(i)
  smallest = i
  if ((l ≤ H.heapsize) and (H.VAL[l] < H.VAL[i]))
    then smallest = l
  if ((r ≤ H.heapsize) and (H.VAL[r] < H.VAL[smallest]))
    then smallest = r
  if (smallest ≠ i)
    then
      {SwapValue(H.VAL, i, smallest)}
      {MinHeapify(H, smallest)}

```

```

proc BuildMinHeap (H)
  {H.heapsize = H.length}
  for (i = ⌊ H.length/2 ⌋ downto 1) MinHeapify(H, i)

```

```

proc HeapSort (H)
  {BuildMaxHeap(H)}
  for (i = H.length downto 2)
    {
      {SwapValue(H.VAL, i, 1)}
      {H.heapsize = H.heapsize - 1}
      {MaxHeapify(H, 1)}
    }

```

```

proc Enqueue (Q, priority)
  if (Q.heapsize = Q.length)
    then return "overflow"
  Q.heapsize = Q.heapsize + 1
  Q.VAL[heapsize] = ∞
  DecreaseKey(Q, Q.heapsize, priority)

```

```

proc DecreaseKey (Q, i, priority)
  if (priority > Q[i])
    then return "error"
  Q.VAL[i] = priority
  while ((i > 1) and (Q.VAL[Parent(i)] > Q.VAL[i]))
    {
      {SwapValue(Q, i, Parent(i))}
      {i = Parent(i)}
    }

```

```

proc ExtractMin (Q)
  if (Q.heapsize < 1)
    then return "underflow"
  min = Q.VAL[1]
  Q.VAL[1] = Q.VAL[Q.heapsize]
  Q.heapsize = Q.heapsize - 1
  MinHeapify(Q, 1)
  return min

```

```

proc Enqueue (Q, i, priority)
  if ((i > Q.length) or (Q.VAL[i] ≠ -1))
    then return "overflow"
  Q.VAL[i] = priority

```

```

proc DecreaseKey (Q, i, priority)
  if ((Q.VAL[i] < priority) or (Q.VAL[i] = -1))
    then return "error"
  Q.VAL[i] = priority

```

```

proc ExtractMin (Q)
  {MinIndex = 0}
  {MinPriority = ∞}
  for (i = 1 to Q.length)
    {
      if ((Q.VAL[i] < MinPriority) and (Q.VAL[i] ≠ -1))
        then
          {
            {MinPriority = Q.VAL[i]}
            {MinIndex = i}
          }
    }
  if (MinIndex = 0)
    then return "underflow"
  Q.VAL[MinIndex] = -1
  return MinIndex

```

```

proc CountingSort (A, B, k)
  let C[0, ..., k] new array
  for (i = 0 to k) C[i] = 0
  for (j = 1 to A.length) C[A[j]] = C[A[j]] + 1
  for (i = 1 to k) C[i] = C[i] + C[i - 1]
  for (j = A.length downto 1)
    {
      {B[C[A[j]]] = A[j]}
      {C[A[j]] = C[A[j]] - 1}
    }

```

```

proc RadixSort (A, d)
{for (i = 1 to d) AnyStableSort(A) on digit i

```

```

proc ListInsert (L, x)
{
  x.next = L.head
  if (L.head ≠ nil)
  then L.head.prev = x
  L.head = x

```

```

proc ListSearch (L, k)
{
  x = L.head
  while (x ≠ nil) and (x.key ≠ k) x = x.next
  return x

```

```

proc ListDelete (L, x)
{
  if (x.prev ≠ nil)
  then x.prev.next = x.next
  else L.head = x.next
  if (x.next ≠ nil)
  then x.next.prev = x.prev

```

```

proc Union (x, y)
{
  S1 = FindSet(x)
  S2 = FindSet(y)
  if (S1 ≠ S2)
  then
    {
      S1.tail.next = S2.head
      z = S2.head
      while (z ≠ nil)
      {
        z.head = S1
        z = z.next
      }
      S1.tail = S2.tail
    }

```

```

proc MakeSet (S, x)
{S[x.key].set = x

```

```

proc FindSet (x)
{return x.head

```

```

proc Union (x, y)
{
  S1 = FindSet(x)
  S2 = FindSet(y)
  if (S1 ≠ S2)
  then
    {
      if (S2.rank > S1.rank)
      then SwapVariable(S1, S2)
      S1.tail.next = S2.head
      z = S2.head
      while (z ≠ nil)
      {
        z.head = S1
        z = z.next
      }
      S1.tail = S2.tail
      S1.rank = S1.rank + S2.rank
    }

```

```

proc Union (x, y)
{
  x = Findset(x)
  y = Findset(y)
  if (x.rank > y.rank)
  then y.p = x
  if (x.rank ≤ y.rank)
  then
    {
      x.p = y
      if (x.rank = y.rank)
      then y.rank = y.rank + 1
    }

```

```

proc FindSet (x)
{
  if (x ≠ x.p)
  then x.p = FindSet(x.p)
  return x.p

```

```

proc HashInsert (T, k)
{
  let x be a new node with key k
  i = h(k)
  ListInsert(T[i], x)

```

```

proc HashSearch (k)
{
  i = h(k)
  return ListSearch(T[i], k)

```

```

proc HashDelete (k)
{
  i = h(k)
  x = ListSearch(T[i], k)
  ListDelete(T[i], x)

```

```

proc OaHashInsert (T, k)
{
  i = 0
  repeat
    {
      j = h(k, i)
      if (T[j] = nil)
      then
        {
          T[j] = k
          return j
        }
      else i = i + 1
    }
  until (i = m)
  return "overflow"

```

```

proc OaHashSearch (T, k)
{
  i = 0
  repeat
    {
      j = h(k, i)
      if (T[j] = k)
      then return j
      i = i + 1
    }
  until ((T[j] = nil) or (i = m))
  return nil

```

```

proc HashComputeModulo (w, B, m)
{
  let d = |w|
  z0 = 0
  for (i = 1 to d) zi+1 = ((zi · B) + ai) mod m
  return zd + 1

```

```

proc TreeInOrderTreeWalk (x)
{
  if (x ≠ nil)
  then
    {
      TreeInOrderTreeWalk(x.left)
      Print(x.key)
      TreeInOrderTreeWalk(x.right)
    }

```

```

proc TreePreOrderTreeWalk (x)
{
  if (x ≠ nil)
  then
    {
      Print(x.key)
      TreePreOrderTreeWalk(x.left)
      TreePreOrderTreeWalk(x.right)
    }

```

```

proc TreePostOrderTreeWalk (x)
{
  if (x ≠ nil)
  then
    {
      TreePostOrderTreeWalk(x.left)
      TreePostOrderTreeWalk(x.right)
      Print(x.key)
    }

```

```

proc BSTTreeSearch (x, k)
{
  if ((x = nil) or (x.key = k))
  then return x
  if (k ≤ x.key)
  then return BSTTreeSearch(x.left, k)
  else return BSTTreeSearch(x.right, k)

```

```

proc BSTTreeMinimum (x)
{
  if ((x.left = nil)
  then return x
  return BSTTreeMinimum(x.left)

```

```

proc BSTTreeSuccessor (x)
{
  if (x.right ≠ nil)
  then return BSTTreeMinimum(x.right)
  y = x.p
  while ((y ≠ nil) and (x = y.right))
  {
    x = y
    y = y.p
  }
  return y

```

```

proc BSTTreeInsert (T, z)
{
  y = nil
  x = T.root
  while (x ≠ nil)
  {
    y = x
    if (z.key ≤ x.key)
    then x = x.left
    else x = x.right
  }
  z.p = y
  if (y = nil)
  then T.root = z
  if ((y ≠ nil) and (z.key ≤ y.key))
  then y.left = z
  if ((y ≠ nil) and (z.key > y.key))
  then y.right = z
}

```

```

proc BSTTreeDelete (T, z)
{
  if (z.left = nil)
  then BSTTreeTransplant(T, z, z.right)
  if ((z.left ≠ nil) and (z.right = nil))
  then BSTTreeTransplant(T, z, z.left)
  if ((z.left ≠ nil) and (z.right ≠ nil))
  then
  {
    y = BSTTreeMinimum(z.right)
    if (y.p ≠ z)
    then
    {
      BSTTreeTransplant(T, y, y.right)
      y.right = z.right
      y.right.p = y
    }
    BSTTreeTransplant(T, z, y)
    y.left = z.left
    y.left.p = y
  }
}

```

```

proc BSTTreeTransplant (T, u, v)
{
  if (u.p = nil)
  then T.root = v
  if ((u.p ≠ nil) and (u = u.p.left))
  then u.p.left = v
  if ((u.p ≠ nil) and (u = u.p.right))
  then u.p.right = v
  if (v ≠ nil)
  then v.p = u.p
}

```

```

proc BSTTreeLeftRotate (T, x)
{
  y = x.right
  x.right = y.left
  if (y.left ≠ T.nil)
  then y.left.p = x
  y.p = x.p
  if (x.p = T.nil)
  then T.root = y
  if ((x.p ≠ T.nil) and (x = x.p.left))
  then x.p.left = y
  if ((x.p ≠ T.nil) and (x = x.p.right))
  then x.p.right = y
  y.left = x
  x.p = y
}

```

```

proc RBTreeInsert (T, z)
{
  y = T.nil
  x = T.root
  while (x ≠ T.nil)
  {
    y = x
    if (z.key < x.key)
    then x = x.left
    else x = x.right
  }
  z.p = y
  if (y = T.nil)
  then T.root = z
  if ((y ≠ T.nil) and (z.key < y.key))
  then y.left = z
  if ((y ≠ T.nil) and (z.key ≥ y.key))
  then y.right = z
  RBTreeInsertFixup(T, z)
}

```

```

proc RBTreeInsertFixup (T, z)
{
  while (z.p.color = RED)
  {
    if (z.p = z.p.p.left)
    then RBTreeInsertFixupLeft(T, z)
    else RBTreeInsertFixupRight(T, z)
  }
  T.root.color = BLACK
}

```

```

proc RBTreeInsertFixupLeft (T, z)
{
  y = z.p.p.right
  if (y.color = RED)
  then
  {
    z.p.color = BLACK
    y.color = BLACK
    z.p.p.color = RED
    z = z.p.p
  }
  else
  {
    if (z = z.p.right)
    then
    {
      z = z.p
      BSTTreeLeftRotate(T, z)
    }
    z.p.color = BLACK
    z.p.p.color = RED
    BSTTreeRightRotate(T, z.p.p)
  }
}

```

```

proc RBTreeInsertFixupRight (T, z)
{
  y = z.p.p.left
  if (y.color = RED)
  then
  {
    z.p.color = BLACK
    y.color = BLACK
    z.p.p.color = RED
    z = z.p.p
  }
  else
  {
    if (z = z.p.left)
    then
    {
      z = z.p
      BSTTreeRightRotate(T, z)
    }
    z.p.color = BLACK
    z.p.p.color = RED
    BSTTreeLeftRotate(T, z.p.p)
  }
}

```

```

proc BreadthFirstSearch (G, s)
{
  for (u ∈ G.V \ {s})
  {
    u.color = WHITE
    u.d = ∞
    u.π = nil
  }
  s.color = GRAY
  s.d = 0
  s.π = nil
  Q = ∅
  Enqueue(Q, s)
  while (Q ≠ ∅)
  {
    u = Dequeue(Q)
    for (v ∈ G.Adj[u])
    {
      if (v.color = WHITE)
      then
      {
        v.color = GRAY
        v.d = u.d + 1
        v.π = u
        Enqueue(Q, v)
      }
    }
    u.color = BLACK
  }
}

```

```

proc DepthFirstSearch (G)
{
  for (u ∈ G.V)
  {
    u.color = WHITE
    u.π = nil
    time = 0
    for (u ∈ G.V)
    {
      if (u.color = WHITE)
      then DepthVisit(G, u)
    }
  }
}

```

```

proc DepthVisit (G, u)
{
  time = time + 1
  u.d = time
  u.color = GRAY
  for (v ∈ G.Adj[u])
  {
    if (v.color = WHITE)
    then
    {
      v.π = u
      DepthVisit(G, v)
    }
  }
  u.color = BLACK
  time = time + 1
  u.f = time
}

```

```

proc CycleDet (G)
{
  cycle = false
  for (u ∈ G.V) u.color = WHITE
  for (u ∈ G.V)
  {
    if (u.color = WHITE)
    then DepthVisitCycle(G, u)
  }
  return cycle
}

```

```

proc DepthVisitCycle (G, u)
{
  u.color = GRAY
  for (v ∈ G.Adj[u])
  {
    if (v.color = WHITE)
    then DepthVisitCycle(G, v)
    if (v.color = GRAY)
    then cycle = true
  }
  u.color = BLACK
}

```

```

proc TopologicalSort (G)
{
  for (u ∈ G.V) u.color = WHITE
  L = ∅
  time = 0
  for (u ∈ G.V)
  {
    if (u.color = WHITE)
    then DepthVisitTS(G, u)
  }
  return L
}

```

```

proc DepthVisitTS (G, u)
{
  time = time + 1
  u.d = time
  u.color = GRAY
  for (v ∈ G.Adj[u])
  {
    if (v.color = WHITE)
    then DepthVisitTS(G, v)
  }
  u.color = BLACK
  time = time + 1
  u.f = time
  ListInsert(L, u)
}

```

```

proc StronglyConnectedComponents (G)
{
  for (u ∈ G.V)
  {
    u.color = WHITE
    u.π = nil
  }
  time = 0
  for (u ∈ G.V)
  {
    if (u.color = WHITE)
    then DepthVisit(G, u)
  }
  GT = Transpose(G)
  for (u ∈ GT.V)
  {
    u.color = WHITE
    u.π = nil
  }
  time = 0
  L = ∅
  for (u ∈ GT.V in rev. finish time order)
  {
    if (u.color = WHITE)
    then DepthVisit(GT, u)
  }
  ListInsert(L, u)
  return L
}

```

```

proc MSTPrim (G, r)
{
  for (v ∈ G.V)
  {
    do
    {
      v.key = ∞
      v.π = nil
    }
    r.key = 0
    Q = G.V
    while (Q ≠ ∅)
    do
    {
      u = ExtractMin(Q)
      for (v ∈ G.Adj[u])
      do
      {
        if ((v ∈ Q) and (W(u, v) < v.key))
        do
        {
          v.π = u
          v.key = W(u, v)
        }
      }
    }
  }
}

```

```

proc MSTKruskal (G)
{
  T = ∅
  for (v ∈ G.V)
  do MakeSet(v)
  SortNoDecreasing(G.E)
  for ((u, v) ∈ G.E - in order)
  do
  {
    if (FindSet(u) ≠ FindSet(v))
    then
    {
      T = T ∪ {(u, v)}
      Union(u, v)
    }
  }
  return A
}

```

```

proc DAGShortestPath (G, s)
{
  TopologicalSort(G)
  InitializeSingleSource(G, s)
  for (u ∈ G.V - in order)
  {
    for (v ∈ G.Adj[u]) Relax(u, v, W)
  }
}

```

```

proc InitalizeSingleSource (G, s)
{
  for (v ∈ G.V)
  {
    v.d = ∞
    v.π = nil
    s.d = 0
  }
}

```

```

proc Relax (u, v, W)
{
  if (v.d > u.d + W(u, v))
  then
  {
    v.d = u.d + W(u, v)
    v.π = u
  }
}

```

```

proc Bellman-Ford (G, s)
{
  InitializeSingleSource(G, s)
  for i = 1 to |G.V| - 1
  {
    for ((u, v) ∈ G.E) Relax(u, v, W)
  }
  for ((u, v) ∈ G.E)
  {
    if (v.d > u.d + W(u, v))
    then return false
  }
  return true
}

```

```

proc Dijkstra (G, s)
{
  InitializeSingleSource(G, s)
  S = ∅
  Q = G.V
  while (Q ≠ ∅)
  {
    u = ExtractMin(Q)
    S = S ∪ {u}
    for (v ∈ G.Adj[u])
    {
      if (v ∈ Q)
      then Relax(u, v, W)
    }
  }
}

```

```

proc PrintPath (Π, i, j)
{
  if (i = j)
  then Print(i)
  if (Πij = nil)
  then Print('nopath')
  else
  {
    PrintPath(Π, i, Πij)
    Print(j)
  }
  return
}

```

```

proc SlowExtendShortestPaths (D, W, Π)
{
  n = D.rows
  let D' be a new matrix
  for i = 1 to n
  {
    for j = 1 to n
    {
      D'ij = ∞
      for k = 1 to n
      {
        D'ij = min{D'ij, Dik + Wkj}
        if (Dij ≠ D'ij)
        then Πij = k
      }
    }
  }
  return D'
}

```

```

proc SlowAllPairsMatrix ( $G$ )
{
   $n = D.rows$ 
   $D^1 = G.W$ 
  let  $\Pi$  be a new matrix
  for  $m = 2$  to  $n - 1$ 
  {
     $D^m = \text{SlowExtendShortestPaths}(D^{m-1}, G.W, \Pi)$ 
  }
  return  $D^{n-1}$ 

```

```

proc FastExtendShortestPaths ( $D, W, \Pi$ )
{
   $n = D.rows$ 
  let  $D'$  be a new matrix
  for  $i = 1$  to  $n$ 
  {
    for  $j = 1$  to  $n$ 
    {
       $D'_{ij} = \infty$ 
      for  $k = 1$  to  $n$ 
      {
         $D'_{ij} = \min\{D'_{ij}, D_{ik} + D_{kj}\}$ 
        if ( $D_{ij} \neq D'_{ij}$ )
        {
          then  $\Pi_{ij} = \Pi_{kj}$ 
        }
      }
    }
  }
  return  $D'$ 

```

```

proc FastAllPairsMatrix ( $G, \Pi^0$ )
{
   $n = L.rows$ 
   $D^1 = G.W$ 
   $m = 1$ 
  let  $\Pi$  be a new matrix
   $\Pi = \Pi^0$ 
  while ( $m < n - 1$ )
  {
    do
    {
       $D^{2 \cdot m} = \text{FastExtendShortestPaths}(D^m, D^m, \Pi)$ 
       $m = 2 \cdot m$ 
    }
  }
  return  $D^m$ 

```

```

proc FloydWarshall ( $G, \Pi^0$ )
{
   $n = W.rows$ 
   $D^0 = G.W$ 
  for  $k = 1$  to  $n$ 
  {
    let  $D^k$  be a new matrix
    let  $\Pi^k$  be a new matrix
    for  $i = 1$  to  $n$ 
    {
      for  $j = 1$  to  $n$ 
      {
         $D^k_{ij} = \min\{D^{k-1}_{ij}, D^{k-1}_{ik} + D^{k-1}_{kj}\}$ 
        if ( $D^k_{ij} \neq D^{k-1}_{ij}$ )
        {
          then  $\Pi^k_{ij} = \Pi^{k-1}_{kj}$ 
        }
      }
    }
  }

```

```

proc kClique ( $G, k$ )
{
  for ( $A \subseteq G.V, |A| = k$ )
  {
     $Clique = \text{true}$ 
    for ( $u, v \in A$ )
    {
      if ( $(u, v) \notin G.E$ )
      {
        then  $Clique = \text{false}$ 
      }
    }
    if ( $Clique = \text{true}$ )
    {
      then return true
    }
  }
  return false

```

```

proc kCliqueCheck ( $G, A, k$ )
{
  if ( $|A| \neq k$ )
  {
    then return false
  }
   $Clique = \text{true}$ 
  for ( $u, v \in A$ )
  {
    if ( $(u, v) \notin G.E$ )
    {
      then  $Clique = \text{false}$ 
    }
  }
  if ( $Clique = \text{true}$ )
  {
    then return true
  }
  return false

```

```

proc HasWinningStrategy ( $G, v$ )
{
  if ( $\text{Empty}(\text{Adj}[v])$ )
  {
    then return false
  }
   $v.mark = \text{true}$ 
   $Wins = \text{true}$ 
  for ( $u \in \text{Adj}[v]$ )
  {
    if ( $u.mark = \text{false}$ )
    {
      then  $Wins = Wins \text{ and not}(\text{HasWinningStrategy}(G, u))$ 
    }
  }
   $v.mark = \text{false}$ 
  return  $Wins$ 

```